Reporte

Nombre:	**Matrícula**:
Angélica Reyna García	2144918
Nohemi Delgado Castillo	2145346
Joselyn María Quiroga Martínez	2144717
Nombre del curso: Componentes de la lógica de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 24	**Actividad**: Unidad 4
Fecha: 28/10/2022	
Bibliografia (Formato APA)	

Objetivo:

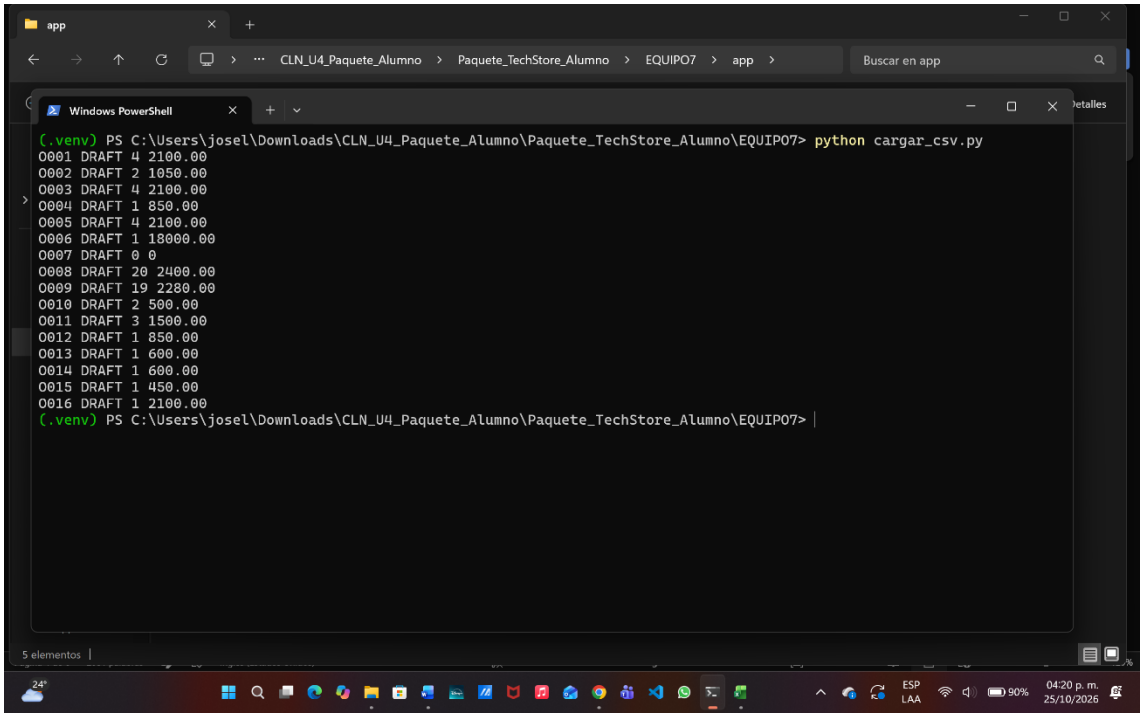
El objetivo de esta actividad es implementar el núcleo de lógica de negocio de una tienda ficticia (TechStore) aplicando principios de diseño orientado al dominio. Para ello, se modelan entidades y objetos de valor (Order, Customer, Money y OrderItem) que protegen sus propias reglas e impiden datos inválidos; se encapsulan las reglas de descuento mediante el patrón Specification, combinadas con operadores AND, OR y NOT; y se controla el ciclo de vida de cada pedido con una máquina de estados que registra Domain Events solo después de transiciones válidas. Finalmente, la lógica se expone mediante una API REST con FastAPI, que devuelve respuestas HTTP apropiadas (404, 409 y 422), y se valida con pruebas automáticas en pytest y con la ejecución de 15 escenarios, de modo que la lógica de negocio quede separada de la interfaz y pueda demostrarse y reutilizarse.

Procedimiento:

Actividad

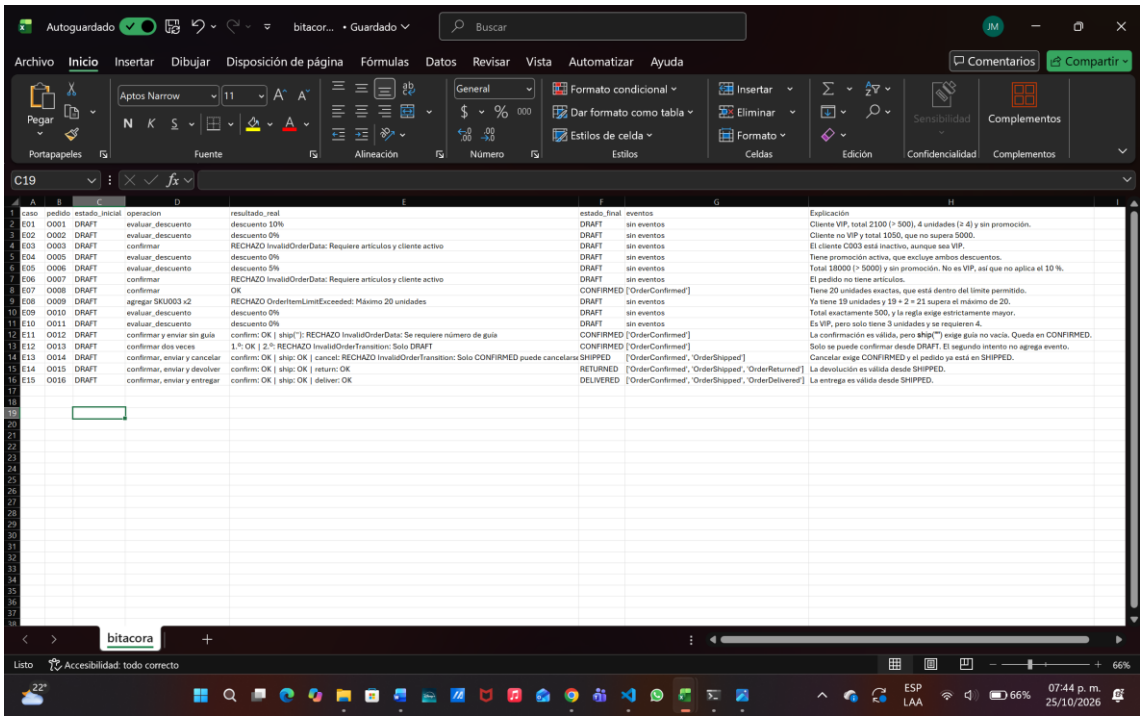
Pedido	Cliente	Unidades	Total manual
O001	C001	4	2,100
O002	C002	2	1.050
O003	C003	4	2,100
O004	C004	1	850
O005	C005	4	2,100
O006	C006	1	18,000
O007	C007	0	0
O008	C008	20	2,400
O009	C009	19	2,280
O010	C010	2	500
O011	C011	3	1,500
O012	C012	1	850
O013	C001	1	600
O014	C002	1	600
O015	C005	1	450
O016	C007	1	2100

Evidencia de carga de los 16 pedidos



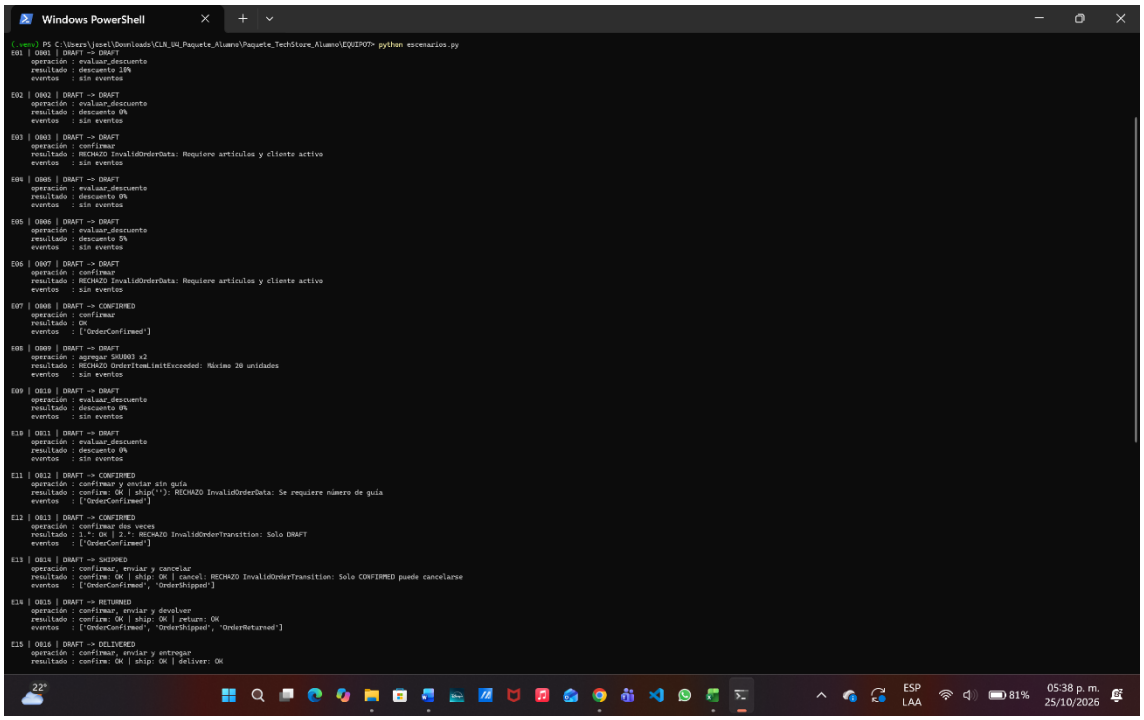
Reporte

Evidencia de bitácora



caso	pedido	estado_inicial	operacion	resultado_real	estado_final	eventos	Explicación
E01	O001	DRAFT	evaluar_descuento	descuento 10%	DRAFT	sin eventos	Cliente VIP, total 2100 (> 500), 4 unidades (≥ 4) y sin promoción.
E02	O002	DRAFT	evaluar_descuento	descuento 0%	DRAFT	sin eventos	Cliente no VIP y total 1050, que no supera 5000.
E03	O003	DRAFT	confirmar	RECHAZO InvalidOrderData: Requiere artículos y cliente activo	DRAFT	sin eventos	El cliente C003 está inactivo, aunque sea VIP.
E04	O005	DRAFT	evaluar_descuento	descuento 0%	DRAFT	sin eventos	Tiene promoción activa, que excluye ambos descuentos.
E05	O006	DRAFT	evaluar_descuento	descuento 5%	DRAFT	sin eventos	Total 18000 (> 5000) y sin promoción. No es VIP, así que no aplica el 10 %.
E06	O007	DRAFT	confirmar	RECHAZO InvalidOrderData: Requiere artículos y cliente activo	DRAFT	sin eventos	El pedido no tiene artículos.
E07	O008	DRAFT	confirmar	OK	CONFIRMED	['OrderConfirmed']	Tiene 20 unidades exactas, que está dentro del límite permitido.
E08	O009	DRAFT	agregar SKU003 x2	RECHAZO OrderItemLimitExceeded: Máximo 20 unidades	DRAFT	sin eventos	Ya tiene 19 unidades y 19 + 2 = 21 supera el máximo de 20.
E09	O010	DRAFT	evaluar_descuento	descuento 0%	DRAFT	sin eventos	Total exactamente 500, y la regla exige estrictamente mayor.
E10	O011	DRAFT	evaluar_descuento	descuento 0%	DRAFT	sin eventos	Es VIP, pero solo tiene 3 unidades y se requieren 4.
E11	O012	DRAFT	confirmar y enviar sin guia	confirm: OK | ship(''): RECHAZO InvalidOrderData: Se requiere número de guía	CONFIRMED	['OrderConfirmed']	La confirmación es válida, pero ship('') exige guía no vacía. Queda en CONFIRMED.
E12	O013	DRAFT	confirmar dos veces	1.ª: OK | 2.ª: RECHAZO InvalidOrderTransition: Solo DRAFT	CONFIRMED	['OrderConfirmed']	Solo se puede confirmar desde DRAFT. El segundo intento no agrega evento.
E13	O014	DRAFT	confirmar, enviar y cancelar	confirm: OK | ship: OK | cancel: RECHAZO InvalidOrderTransition: Solo CONFIRMED puede cancelarse	SHIPPED	['OrderConfirmed', 'OrderShipped']	Cancelar exige CONFIRMED y el pedido ya está en SHIPPED.
E14	O015	DRAFT	confirmar, enviar y devolver	confirm: OK | ship: OK | return: OK	RETURNED	['OrderConfirmed', 'OrderShipped', 'OrderReturned']	La devolución es válida desde SHIPPED.
E15	O016	DRAFT	confirmar, enviar y entregar	confirm: OK | ship: OK | deliver: OK	DELIVERED	['OrderConfirmed', 'OrderShipped', 'OrderDelivered']	La entrega es válida desde SHIPPED.

Evidencia de escenarios de la terminal



```
(.venv) PS C:\Users\josel\Downloads\CLN_U4_Paquete_Alumno\Paquete_TechStore_Alumno\EQUIPO7> python escenarios.py
E01 | O001 | DRAFT -> DRAFT
     operación : evaluar_descuento
     resultado : descuento 10%
     eventos   : sin eventos

E02 | O002 | DRAFT -> DRAFT
     operación : evaluar_descuento
     resultado : descuento 0%
     eventos   : sin eventos

E03 | O003 | DRAFT -> DRAFT
     operación : confirmar
     resultado : RECHAZO InvalidOrderData: Requiere artículos y cliente activo
     eventos   : sin eventos

E04 | O005 | DRAFT -> DRAFT
     operación : evaluar_descuento
     resultado : descuento 0%
     eventos   : sin eventos

E05 | O006 | DRAFT -> DRAFT
     operación : evaluar_descuento
     resultado : descuento 5%
     eventos   : sin eventos

E06 | O007 | DRAFT -> DRAFT
     operación : confirmar
     resultado : RECHAZO InvalidOrderData: Requiere artículos y cliente activo
     eventos   : sin eventos

E07 | O008 | DRAFT -> CONFIRMED
     operación : confirmar
     resultado : OK
     eventos   : ['OrderConfirmed']

E08 | O009 | DRAFT -> DRAFT
     operación : agregar SKU003 x2
     resultado : RECHAZO OrderItemLimitExceeded: Máximo 20 unidades
     eventos   : sin eventos

E09 | O010 | DRAFT -> DRAFT
     operación : evaluar_descuento
     resultado : descuento 0%
     eventos   : sin eventos

E10 | O011 | DRAFT -> DRAFT
     operación : evaluar_descuento
     resultado : descuento 0%
     eventos   : sin eventos

E11 | O012 | DRAFT -> CONFIRMED
     operación : confirmar y enviar sin guia
     resultado : confirm: OK | ship(''): RECHAZO InvalidOrderData: Se requiere número de guía
     eventos   : ['OrderConfirmed']

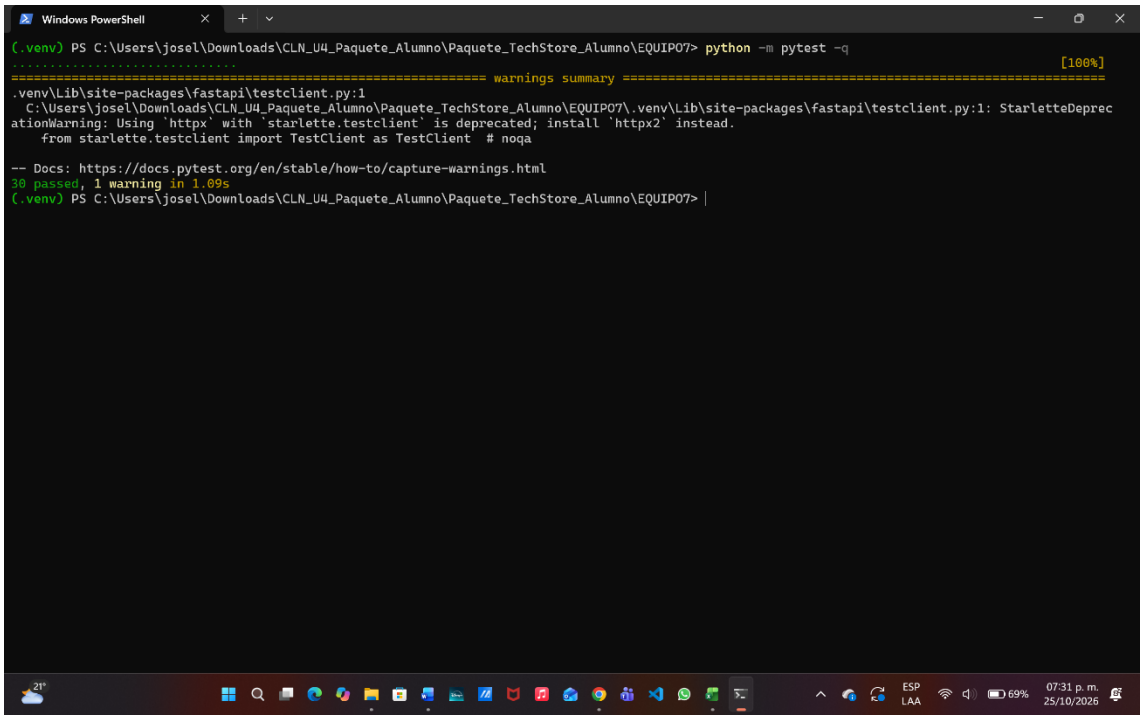
E12 | O013 | DRAFT -> CONFIRMED
     operación : confirmar dos veces
     resultado : 1.ª: OK | 2.ª: RECHAZO InvalidOrderTransition: Solo DRAFT
     eventos   : ['OrderConfirmed']

E13 | O014 | DRAFT -> SHIPPED
     operación : confirmar, enviar y cancelar
     resultado : confirm: OK | ship: OK | cancel: RECHAZO InvalidOrderTransition: Solo CONFIRMED puede cancelarse
     eventos   : ['OrderConfirmed', 'OrderShipped']

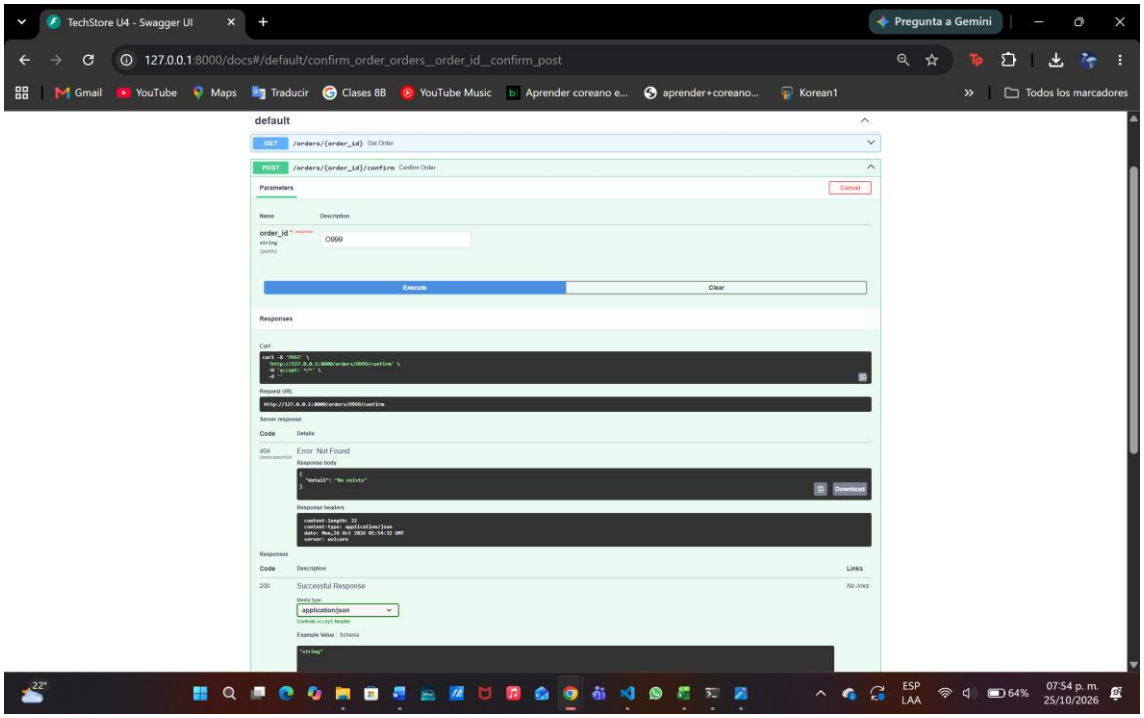
E14 | O015 | DRAFT -> RETURNED
     operación : confirmar, enviar y devolver
     resultado : confirm: OK | ship: OK | return: OK
     eventos   : ['OrderConfirmed', 'OrderShipped', 'OrderReturned']

E15 | O016 | DRAFT -> DELIVERED
     operación : confirmar, enviar y entregar
     resultado : confirm: OK | ship: OK | deliver: OK
     eventos   : ['OrderConfirmed', 'OrderShipped', 'OrderDelivered']
```

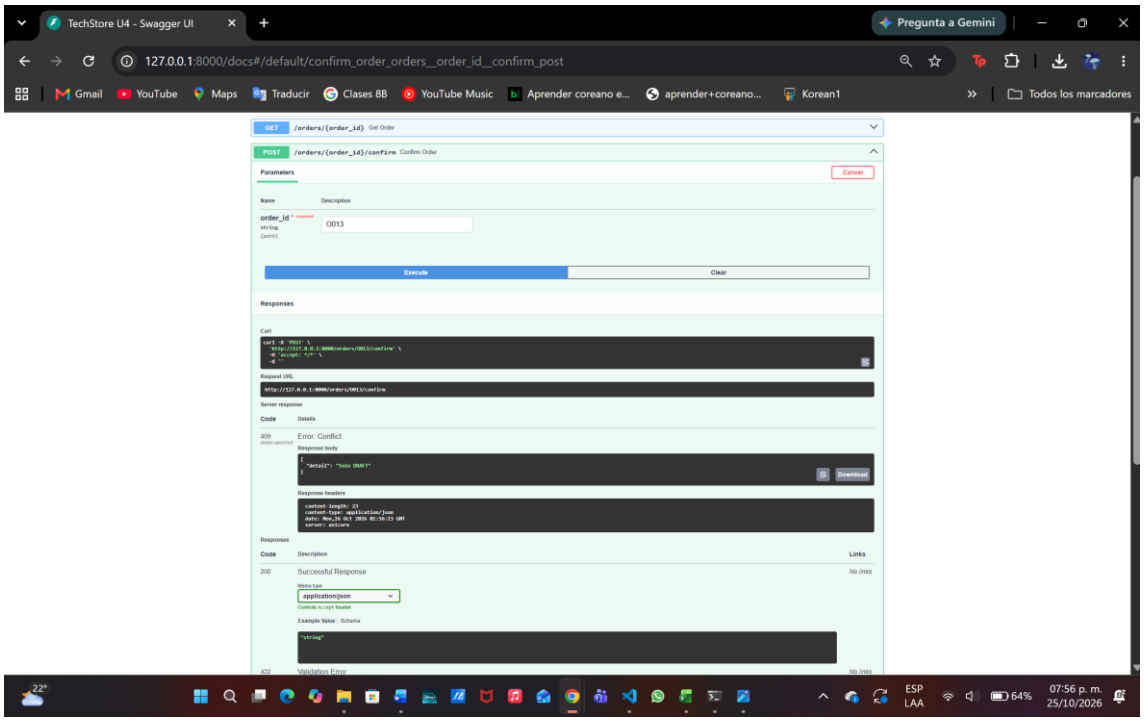
Evidencia de escenarios de la pytest



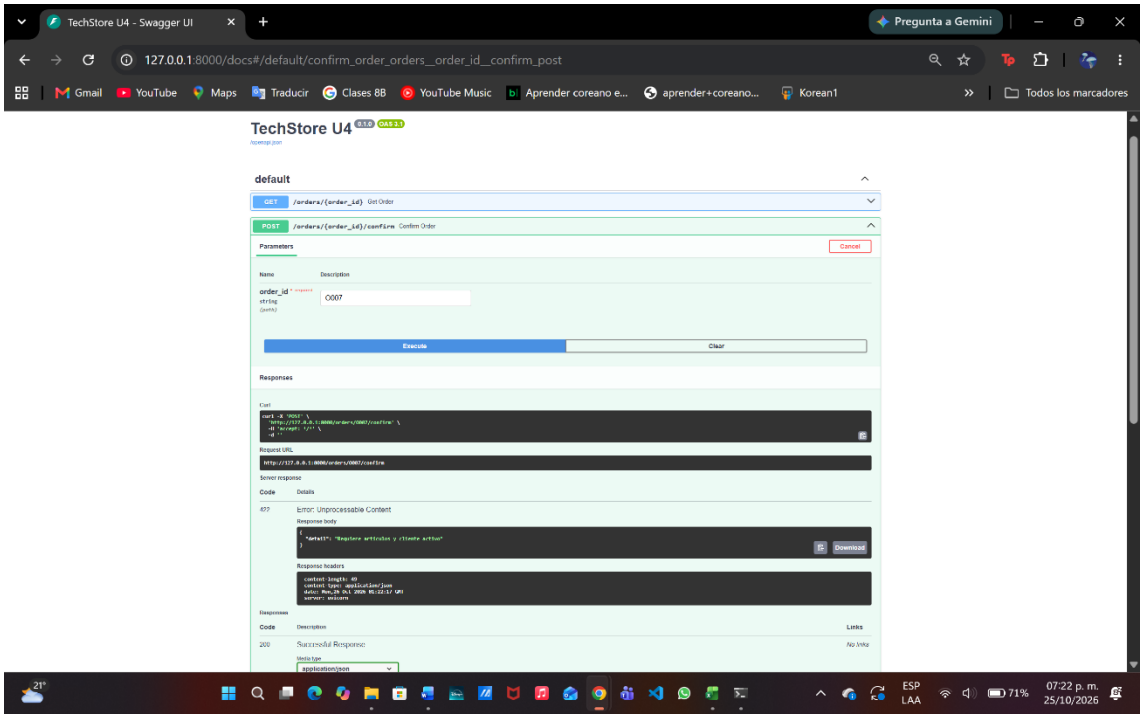
Evidencia de los swagger



404


TechStore U4 - Swagger UI
Pregunta a Gemini
127.0.0.1:8000/docs#/default/confirm_order_orders__order_id__confirm_post
/orders/{order_id} Get Order
/orders/{order_id}/confirm Confirm Order
Parameters
Cancel
order_id
0013
Execute
Clear
Responses
Error: Conflict
Response body
Download
Response headers
Code
Description
Links
200
Successful Response
No links
application/json
Example Value | Schema
"string"
422
Validation Error

409


TechStore U4 - Swagger UI
Pregunta a Gemini
127.0.0.1:8000/docs#/default/confirm_order_orders__order_id__confirm_post
TechStore U4
default
/orders/{order_id} Get Order
/orders/{order_id}/confirm Confirm Order
Parameters
Cancel
order_id
0007
Execute
Clear
Responses
Error: Unprocessable Content
Response body
Download
Response headers
Code
Description
Links
200
Successful Response
No links
application/json

422

Reflexión breve: tres errores encontrados, cómo los solucionamos y qué aprendimos.

1. Pegar código de Python en la terminal. Al principio pegamos el contenido de domain.py directamente en PowerShell, y la terminal respondió con errores como que raise no se reconocía como un comando. Entendimos que la terminal solo sirve para ejecutar comandos, y que el código de Python debe guardarse en un archivo .py para después ejecutarlo con python archivo.py. Lo solucionamos guardando el código en app/domain.py, y a partir de ahí cargar_csv.py pudo importar las clases.

2. Bloqueo de scripts al activar el entorno virtual. Al abrir una terminal nueva, PowerShell no dejó activar .venv porque la ejecución de scripts estaba deshabilitada. Lo resolvimos permitiendo los scripts solo en esa sesión con Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, sin cambiar la configuración permanente de la computadora. Aprendimos que el entorno virtual hay que activarlo cada vez que se abre una terminal nueva, porque ahí están instaladas las dependencias.

3. Confundir la letra O con el número cero en los IDs. En Swagger, las primeras pruebas dieron 404 aunque el pedido sí existía, porque escribimos 0013 con cero en lugar de O013 con la letra O. También escribimos 409 y 422 como si fueran IDs, cuando son códigos de respuesta. Lo corregimos revisando la línea "Request URL" de cada respuesta, que mostraba exactamente qué ID se había enviado, y usando IDs reales: O999 para el 404, O013 dos veces para el 409 y O007 para el 422.

Con la práctica aprendimos que las reglas de negocio deben vivir en el dominio y no en la interfaz: la API solo busca el pedido, llama al método y traduce el error a un código HTTP, mientras que el dominio decide qué es válido. También vimos la importancia de usar Decimal para dinero, de validar antes de cambiar el estado, y de que las pruebas automáticas, junto con los 15 escenarios, nos permitieron comprobar que las reglas funcionan y que fallan correctamente cuando deben. Por último, aprendimos que cuando una prueba falla hay que corregir la implementación y no el resultado esperado.

Conclusiones

Angélica Reyna García

Para esta actividad fue importante conocer la identidad de TechStore que hemos estado trabajando desde unidades anteriores, como definir sus parámetros, restricciones y aprender a identificar los vacíos para asegurarnos de que el producto entregable fuera lo menos des prolijo posible. También fue crucial entender que función cumple cada parte como los diagramas bpmn y cómo conectarlos con nuestro código en Python, de esta forma obtenemos una unidad 4 la cual es funcional y coherente con todas sus partes.

Nohemi Delgado Castillo

A lo largo de esta actividad aprendí a aplicar los principios del Diseño Orientado al Dominio para construir un sistema sólido, bien estructurado y fácil de mantener. Este proceso me enseñó a priorizar las reglas reales del negocio desde la base, creando elementos que protegen su propia información para evitar datos incorrectos y utilizando estructuras flexibles para gestionar la lógica y el flujo de los procesos sin complicaciones. Además, comprendí el valor de separar por completo la lógica principal de la interfaz web y de validar cada caso mediante pruebas automáticas, lo que me permitió comprobar que diseñar con un enfoque centrado en las necesidades del problema facilita la escalabilidad y garantiza la calidad del software.

Joselyn María Quiroga Martínez

Con esta práctica comprendí que la lógica de negocio debe ser la parte más cuidada e importante de un sistema, porque ahí viven las reglas que lo hacen funcionar. Al principio construir clases como Money, OrderItem y Order con tantas validaciones me pareció más complicado que guardar datos, pero al ejecutar los escenarios vi su valor: el sistema no permite montos negativos, pedidos de 21 unidades ni confirmar dos veces el mismo pedido, y los casos frontera (como un total exactamente de 500) fueron los que más me enseñaron. También entendí por qué conviene separar el dominio de la interfaz: los endpoints de FastAPI solo llaman al método y traducen el error a un código HTTP, así que las reglas pueden probarse con pytest y reutilizarse sin depender de la API. Además, el patrón Specification me pareció útil porque permite leer una regla de descuento casi como se describe en el negocio, y los errores que tuve (pegar código en la terminal, activar el entorno virtual o confundir la letra O con el cero) me enseñaron a leer los mensajes de error con calma y a verificar lo básico primero.